

Lesson 3 Stock Transfer



This lesson is customized for users who have bought the deluxe version.

1. Program Logic

As logistics industry advances swiftly, goods in warehouse are increasing frequently entered, delivered and transferred. In order to facilitate the management and enhance the efficiency and accuracy of goods transferring, we use robot arm to transfer goods between shelves, which optimizes the shelves arrangement to ensure accurate, safe and efficient internal operations.

In this lesson, we are going to take a look at stock transfer function of ArmPi FPV.

First of all, we need to set the shelf number of the item to be transferred and the shelf number the item to be warehoused in the program. Then set the external input to decide the shelf and floor location before and after the goods transferring.

After receiving the data, transfer subroutine will control the body of robotic arm to turn to the corresponding shelf. After the picking angle and height is adjusted by transfer subroutine, the robotic arm will approach and pick up the goods. Next, control the robotic arm to transfer and place the goods according to the shelf numbers set in the program.

The source code is located in
`/home/armpi_fpv/src/warehouse/scripts/exchange.py`.

```
24 # transform
25 # If not declared, the length and distance units used are all m
26
27 # Initialization
28 __target_data = ()
29 __isRunning = False
30 org_image_sub_ed = False
31 lock = threading.RLock()
32 ik = ik_transform.ArmIK()
33
34 # initial position
35 def initMove():
36     with lock:
37         bus_servo_control.set_servos(joints_pub, 1500, ((1, 75
38             ), (2, 500), (3, 80), (4, 825), (5, 625), (6, 500)))
39         rospy.sleep(2)
40
41 # reset
42 def reset():
43     global __target_data
44     __target_data = ()
45
46 # app initialization
47 def init():
48     rospy.logininfo("exchange Init")
```

2. Operation Steps

i The entered command must be strictly distinguished between uppercase and lowercase and spaces, and the keywords support the "TAB" key completing.

Let's take the example of making the robotic arm transfer the goods on the second floor of the left shelf to the third floor of the right shelf.

2.1 Preparation

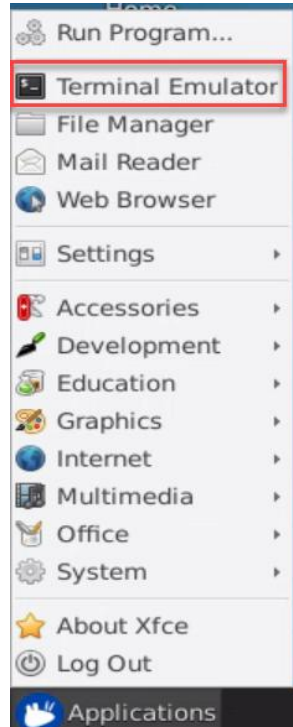
① Please place the map on even desk, and the robotic arm and shelf should be placed on the corresponding area on the map. And prepare a block and place it on the center of the second floor.

② Turn on the robotic arm and wait to finish.

2.2 Enter the Game

① Turn on ArmPi FPV and connect to Raspberry Pi desktop through NoMachine.

② Click “Applications” button and select “Terminal Emulator” in pop-up interface to open the command line terminal.



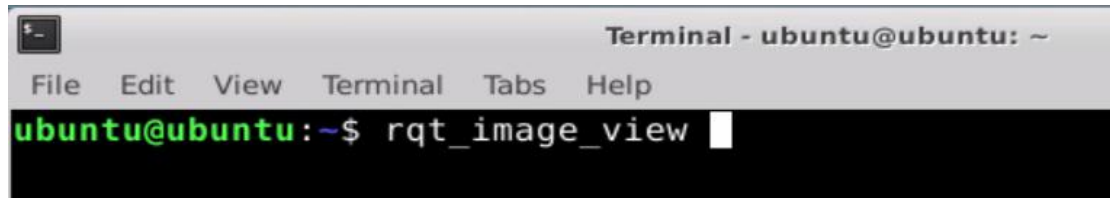
③ Enter “**rosservice call /exchange/enter "{}"**” command, then press “Enter” to start the game. After the game starts, a print prompt will appear, as shown in the figure below.

```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ rosservice call /exchange/enter "{}"  
success: True  
message: "enter"  
ubuntu@ubuntu:~$
```

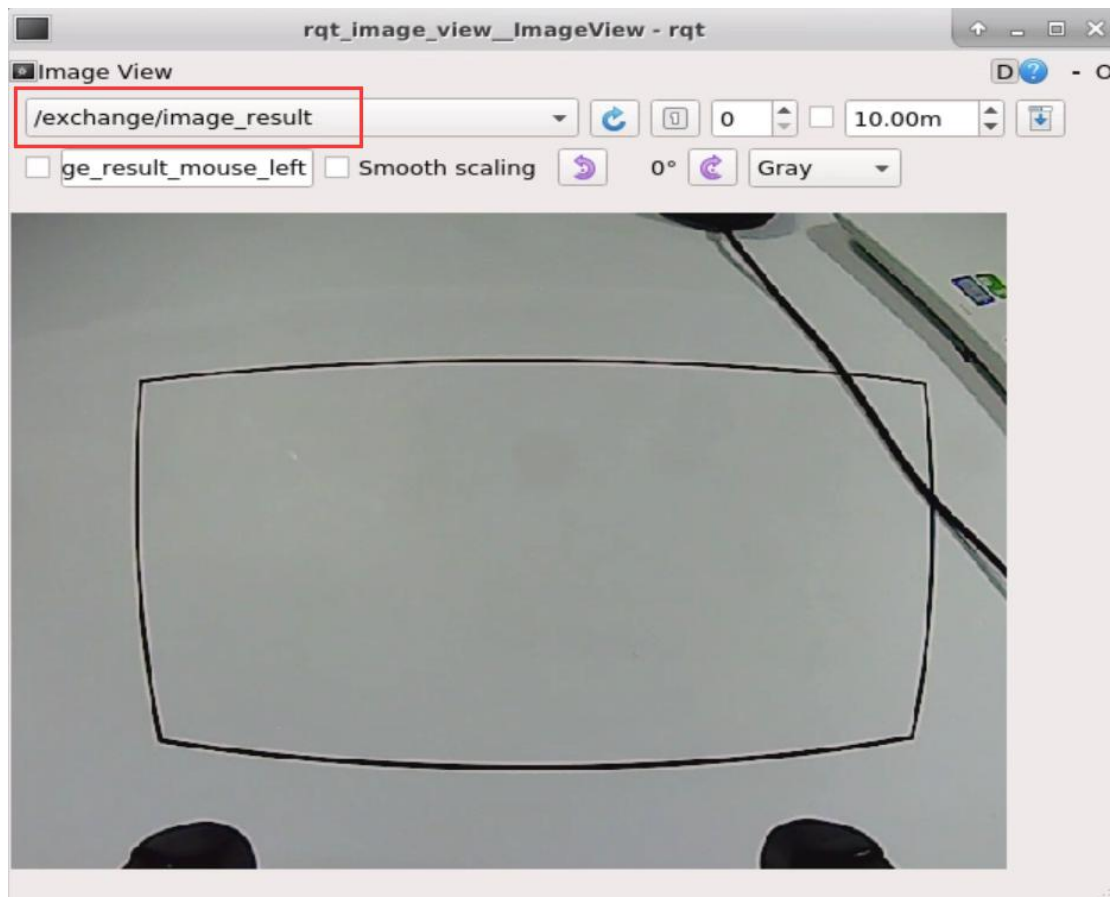
2.3 Turn On the Image

① When entering the game, we need to turn on image returned by camera with rqt. Please go back to the desktop and click the “Application” icon at bottom-left corner to open a new terminal.

② Enter “**rqt_image_view**” command and press “Enter”. Then open rqt after few seconds.



③ As shown in the following figure, select “**/exchange/image_result**” and other settings remain the same.



Note: after the image is turned on, please be sure to select the topic corresponding to the game you play. Otherwise, the recognition process can not be displayed normally if you start other games.

2.4 Start the Game

① Please go back to the terminal opened in “2.2 Enter the Game” and enter **rosservice call /exchange/set_running "data: true"** command. When the prompt, shown in the red box, appears, it means you start the game successfully.

```
ubuntu@ubuntu:~$ rosservice call /exchange/set_running "data: true"
success: True
message: "set_running"
ubuntu@ubuntu:~$
```

② When the game starts, we need to set the parameter, numbers of the shelf. Let's take the example of transferring the goods on the second floor of the left shelf to the third floor of the right shelf.

```
rosservice call /exchange/set_target "position:
- 'L2'
- R3"
```

③ Sets the type of input parameter in the parameter command, and you can refer to the table below for their meanings and built-in parameters.

Type	Meaning	Built-in parameter
position	Numbers of shelves. The first parameter refers to the shelf transferred out. And the second parameter is for the shelf transferred in.	Left Shelf (from bottom to top): L1, L2, L3 Right Shelf (from bottom to top): R1, R2, R3

④ Pay attention to the following points when entering parameters.

(1) As stock transfer is to transfer the goods from one shelf to other shelf, you only need to enter two parameters in this program. And parameters are

separated by a newline. Please strictly follow the correct input form, short bar + space + 'shelf number'.

(2) In order to avoid errors caused by incorrect input method, it is recommended to use the Tab key completing, and enter the parameters in the corresponding position.

(3) If you enter the parameters after Tab key completing, then you need to delete the content after the first parameter before wrapping. After all the parameters are entered, add double quotation marks after the last parameter. The instruction is as follow.

```
ubuntu@ubuntu:~$ rosservice call /exchange/set_target "position:  
- ' " 1
```

```
ubuntu@ubuntu:~$ rosservice call /exchange/set_target "position:  
- 'L2'  
> - 'R3' 2
```

```
ubuntu@ubuntu:~$ rosservice call /exchange/set_target "position:  
- 'L2'  
> - 'R3' " 3
```

2.5 Pause and Quit the Game

You can enter “**rosservice call /exchange/set_running "data: false"**” command to pause the game.

```
ubuntu@ubuntu:~$ rosservice call /exchange/set_running "data: false"  
success: True  
message: "set_running"  
ubuntu@ubuntu:~$
```

Referring to “2.4 Start the game”, you can change the floor of the shelves.

You can enter “**rosservice call /exchange/exit "{}"**” command to quit the game.

```
ubuntu@ubuntu:~$ rosservice call /exchange/exit "{}"  
success: True  
message: "exit"  
ubuntu@ubuntu:~$
```

Note: the current game will continue as Raspberry Pi is powered on. To avoid occupying too much RAM, please quit the current game according to the instruction above before playing other AI vision game.

If you want to turn off the image returned by camera, you can turn back to open rqt terminal and press "Ctrl+C".

3. Function Realization

When the game starts, the robotic arm will transfer the items on the second floor of the left shelf to the third floor of the right shelf.